

AeroNexus Mapping Tool – Architektur- und Machbarkeitsstudie (Stand: 2. Oktober 2026)

Das Modul ist machbar und sollte so gebaut werden: Der Raspberry Pi 5 bleibt Auftragsverwaltung, Datenhaltung und Kartenauslieferung. Ein x64-Rechenknoten holt sich Aufträge selbst ab (Pull-Modell), rechnet sie mit einem fest gepinnten ODM-/NodeODM-Container, schreibt Cloud Optimized GeoTIFFs und vorberechnete Kacheln zurück nach MinIO und meldet den Status. Leaflet bleibt für das MVP erhalten. Die größten Risiken liegen nicht bei der Photogrammetrie. Sie liegen erstens bei der Kompatibilität selbst erzeugter WPML-Mapping-Templates mit Pilot 2 (vor allem Matrice 4), zweitens bei radiometrisch belastbaren Multispektral- und Thermal-Ergebnissen und drittens bei der Aufspaltung des ODM-Ökosystems im April 2026.

TL;DR

- **Architektur:** Der Pi 5 (ARM64) übernimmt Auftrags-API, Warteschlange, MinIO, Web-UI und die Auslieferung statischer Kacheln. Der x64-Knoten (Docker, optional NVIDIA) betreibt einen kleinen „Compute-Agent“. Dieser fragt den Pi ab, lädt die Bilder über vorsignierte S3-URLs, steuert lokal NodeODM per REST, wandelt die Ergebnisse in COG/PMTiles um und lädt sie nach MinIO hoch. Wake-on-LAN ist optional. Weil der Knoten nur ausgehend zieht, darf er jederzeit ausgeschaltet sein.
- **Verarbeitung:** Basis ist ODM 3.6.x (AGPL-3.0, Docker `opendronemap/odm:3.6.1-gpu` bzw. 3.6.2). Die GPU beschleunigt nur die SIFT-Merkmalsextraktion, der Engpass ist der Arbeitsspeicher. Die Tabelle „Minimum RAM needed for N images“ der OpenDroneMap-3.6.0-Doku nennt als Mindestwert („RAM or RAM + Swap“) 16 GB für 250 Bilder, 32 GB für 500 und 64 GB für 1500. Empfohlen werden 64 GB RAM mit großem NVMe-Swap und eine RTX-Karte mit mindestens 8 GB VRAM. Multispektral (M3M) wird seit ODM 3.5.3 unterstützt, die absolute Reflektanz ist aber fraglich. Thermal funktioniert nur über eine Vorverarbeitung mit dem proprietären DJI Thermal SDK v1.8 (nur x86/x64).
- **Vorgehen:** Phase 1 ist das MVP „Bilder aus MinIO auswählen → Orthofoto/DSM auf x64 rechnen → als Layer in Leaflet“. Danach folgen Messwerkzeuge und GCP/RTK, dann eine Offline-Basiskarte (PMTiles) und amtliche Layer (basemap.de, DOP, DGM). Erst danach kommen eine eigene WPML-Flugplanung, Multispektral/Thermal und 3D. Mapping-KMZs müssen vor jedem produktiven Einsatz gegen echte Pilot-2-Exporte jedes Modells validiert werden.

Key Findings

1. **Das ODM-Ökosystem hat sich am 6. April 2026 gespalten.** WebODM hat sich laut der Ankündigung „WebODM has officially decoupled from OpenDroneMap“ von Piero Toffanin vom 6. April 2026 wegen „irreconcilable differences with my co-founder“ von OpenDroneMap getrennt: „If you use ODM or NodeODM, you can find updated forks at ODX and NodeODX, for which stable docker images will continue to receive

updates and improvements at webodm/odx and webodm/nodeodx.“ OpenDroneMap pflegt ODM/NodeODM weiter (ODM 3.6.1 und danach 3.6.2, NodeODM 3.6.2).

(Wikipedia) (zenodo) Beide Seiten bieten dieselbe NodeODM-REST-API, die Docker-Tags laufen aber künftig auseinander. (aerocartwright) (aerocartwright) **Konsequenz:** AeroNexus koppelt sich nur an die NodeODM-API (v2), pinnt genau eine Engine-Linie per Image-Digest und hält die andere als austauschbare Alternative vor.

2. **GPU-Images sind derzeit ein Stolperstein.** Ein NodeODM-Issue (#271) dokumentiert Folgendes: (opendronemap/nodeodm:gpu) wurde seit dem 22.10.2025 nicht neu gebaut, und GPU-SIFT fällt darin wegen einer fehlenden CUDA-11-Laufzeit „silently“ auf die CPU zurück („the job simply runs several times slower“). Nach einem Neubau auf der aktuellen Basis (opendronemap/odm:gpu) (CUDA 12.9, Ubuntu 24.04) meldete ein Tester „popsift can handle texture size 4032x3024 ... True“. (github) **Konsequenz:** Das NodeODM-GPU-Image selbst bauen (aus gepinntem (odm:3.6.x-gpu)) oder ODM direkt per CLI aufrufen. Die GPU-Nutzung muss im Health-Check geprüft werden.
3. **ODM 3.6.1 ist die erste installierbare Version seit 3.5.6.** Sie bringt eine Basis aus Ubuntu 24.04, Python 3.12, aktualisiertem CUDA und GDAL 3.11.1 sowie obj2tiles 1.4.0 (3D Tiles). 3.6.0 wurde nie als Image veröffentlicht. (newreleases)
4. **WPML ist für M3E/M3T/M3M gut dokumentiert, für Matrice 4 nur lückenhaft.** Öffentlich dokumentiert sind Namespace (http://www.dji.com/wpmz/1.0.2) und die Templatetypen (waypoint), (mapping2d), (mapping3d) und (mappingStrip). (github +2) Die von Pilot 2 erzeugten Typen (mappingPrism) und (mappingCylinder) stehen nicht in der öffentlichen Spezifikation. Für Matrice 4 sind die Enum-Werte nur sekundär belegt, der Kamera-Enum des M4E gar nicht (Details unten).
5. **Offline-Karten und amtliche Daten sind gut lösbar.** basemap.de ist CC BY 4.0, wird als WMTS in EPSG:3857, 25832 und 25833 angeboten und quartalsweise aktualisiert. Bayern stellt DGM1 unter CC BY 4.0 bereit. OSM-Vektorkacheln für Deutschland lassen sich mit (pmtiles extract) aus dem Protomaps-Build schneiden. (github +3)
6. **TiTiler läuft inzwischen auch auf ARM64.** Die aktuelle Version ist 2.4.0 vom 21.09.2026, und die Release Notes nennen „add linux/arm64 docker image“. (developmentseed) Für das MVP genügen auf dem Pi aber statische Kacheln aus MinIO. Das ist robuster und spart CPU.

Details

1. Funktionsumfang und WPML-Flugplanung

Sinnvoller Funktionsumfang (Zielbild):

- Flächenraster (mapping2d) mit Überlappung längs/quer, GSD-Rechner, Flugrichtung, Rand ((margin)), Fotomodus Zeit/Distanz und Höhenoptimierung ((elevationOptimizeEnable): am Ende ein Satz Schrägbilder in der Gebietsmitte). (github)

- Schrägaufnahmen (mapping3d, 5-Richtungs-Muster) und Smart Oblique (`smartObliqueEnable` und `smartObliqueGimbalPitch`, bei dem der Gimbal schwenkt). ([github](#)) ([megadron](#))
- Korridor (mappingStrip) für Straßen, Leitungen und Gewässer.
- Fassade bzw. Hang (`facadeWaylineEnable`) mit einem LinearRing im Raum, laut Spezifikation **nur M3E/M3T/M3M**. ([github](#))
- Geländefolge (`surfaceFollowModeEnable` und `surfaceRelativeHeight`), nur für mapping2d/mapping3d/mappingStrip). Der Höhenmodus `realTimeFollowSurface` ist laut Spezifikation „Only supported by M3E/M3T/M3M“. ([github](#))

GSD-Formel (für den eigenen Rechner): $GSD [cm/px] = (Sensorbreite [mm] \times Flughöhe [m] \times 100) / (Brennweite [mm] \times Bildbreite [px])$. Bildabstand längs = Bildhöhe am Boden $\times (1 - \text{Überlappung längs})$. Linienabstand = Bildbreite am Boden $\times (1 - \text{Überlappung quer})$. Die Kameradaten (4/3-Zoll-Sensor, 20 MP beim M3E/M3M-RGB usw.) sollten als versionierte Profile in einer JSON-Datei liegen und nicht im Code stehen. Die DJI-Spezifikation empfiehlt, im `template.kml` Fotomodus, Überlappung und Geschwindigkeit festzulegen. Daraus berechnet der Client das Intervall in `waylines.wpml` („It is recommended to use "time"...“). ([github](#))

Belegte Enum-Werte (für `droneInfo`/`payloadInfo`):

Modell	droneEnumValue/Sub	payloadEnumValue	Beleglage
Mavic 3E	77 / 0	66	DJI Cloud-API-Doc (GitHub-Spiegel) (github +2)
Mavic 3T	77 / 1	67	DJI Cloud-API-Doc (github +2)
Mavic 3M	77 / 2	68	nur WPML-Common-Element-Tabelle (github)
Matrice 4E	99 / 0	88 (?)	nur sekundär , 88 unbestätigt (github) (github)
Matrice 4T	99 / 1	89	nur sekundär (github) (github)

Die Live-Doku von DJI verweist für die Zahlen inzwischen nur noch auf die Seite „Product Support“. M4E/M4T sind dort bei `payloadInfo`, `payloadParam` und `waylineCoordinateSysParam` gelistet. ([dji](#) +2) Für Templatetyp, Smart Oblique, Fassade, `mappingHeadingParam` und Geländefolge ließ sich kein M4-Eintrag bestätigen.

Konsequenz: Die M4-Unterstützung des eigenen Generators gilt als „experimentell“, bis echte KMZ-Exporte aus Pilot 2 (RC Plus 2) vorliegen und als Golden Files im Testkorpus abgelegt sind.

Empfohlene Strategie zur Pilot-2-Kompatibilität: Ein Mapping-KMZ enthält ein `template.kml` (Planungsabsicht) und ein `waylines.wpml` (Ausführung). [\(dji\)](#) Pilot 2 erzeugt aus einem Mapping-Template die Ausführungsrouten selbst. [\(github\)](#) [\(dji\)](#) Daher wird empfohlen, **für Mapping primär nur korrekte `template.kml`-Mapping-Templates zu erzeugen** und die Routenberechnung Pilot 2 zu überlassen. Ein eigenes `waylines.wpml` ist nur für Wegpunkt-Templates und als Vorschau gedacht. Das DJI-MSDK-Tutorial warnt: Ein KMZ mit ungültigem `template.kml` „can not be edited by DJI Pilot“. [\(dji\)](#) Validierung über Golden-File-Tests: echte Pilot-2-Exporte je Modell und Templatetyp ablegen, eigene Ausgabe strukturell vergleichen (XSD-ähnliche Prüfung und Diff) und unbekannte Elemente beim Import erhalten.

Mechanischer Verschluss beim M3E: In der öffentlichen WPML-Spezifikation fand sich dafür kein eigenes Element. Es handelt sich um eine Kameraeinstellung in Pilot 2 und nicht um einen Routenparameter. Der Planer sollte nur passende Geschwindigkeiten und Intervalle für die Belichtung vorschlagen und darauf hinweisen.

Open-Source-Vorbilder für WPML-Export:

- FlyPath (QGIS-Plugin, Export für DJI Fly/RC2) [\(github\)](#)
- open-flight-planner (MIT, Browser, MapLibre, Pilot-2-Export). Laut Selbstauskunft gilt: „Status: not yet flown“. [\(github\)](#)
- DroneRoute (React/TypeScript, self-hosted) [\(github\)](#)
- dji-mission-planner (Korridor, Fassade, Orbit, WPML für Pilot 2) [\(github\)](#)
- DJI-3D-Flight-Planner (TypeScript-Exporter) [\(github\)](#)

Diese Projekte eignen sich als Code-Referenz, ersetzen aber keine Feldvalidierung. Vor einer Übernahme von Code die Lizenz jedes Repos prüfen.

2. Photogrammetrie auf x64

Engine-Wahl: ODM bzw. ODX, gesteuert über die NodeODM- bzw. NodeODX-API. ClusterODM/ClusterODX lohnt sich erst ab mehreren Rechenknoten oder für verteiltes Split-Merge. [\(github\)](#) [\(webodm\)](#) Bei einem einzelnen Knoten ist das unnötige Komplexität. WebODM wird **nicht** integriert, weil es eine eigene Django-App mit eigener Benutzer- und Projektverwaltung ist. [\(aerocartwright\)](#) [\(docker\)](#) Einzelne UI-Ideen wie Plant-Health-Paletten oder das Volumenwerkzeug dienen nur als Vorbild.

NodeODM-REST-API (für den Agenten):

- `GET /info` und `GET /options` (Optionen dynamisch abfragen, nicht hart codieren) [\(deepwiki\)](#)
- `POST /task/new/init` → `POST /task/new/upload/{uuid}` (in Blöcken) → `POST /task/new/commit/{uuid}` [\(deepwiki\)](#) [\(github\)](#)
- `GET /task/{uuid}/info` (Status, Fortschritt) und `GET /task/{uuid}/output` (Log) [\(deepwiki\)](#)

- `GET /task/{uuid}/download/{asset}` (z. B. `all.zip`, `orthophoto.tif`) ([deepwiki](#))
- `POST /task/cancel|restart|remove` ([deepwiki](#))
- Authentifizierung per Token als Query-Parameter. ([github](#))
- Python-Alternative: PyODM mit `create_task(files, options, ...)`. ([readthedocs](#))

GPU: Laut ODM-README gilt: „ODM has support for doing SIFT feature extraction on a GPU, which is about 2x faster than the CPU“. ([aerocartwright](#)) Die Implementierung ist CUDA-basiert (ab GTX 9xx). ([github](#)) Die offizielle Hardware-Seite sagt dagegen weiterhin, eine GPU habe „currently no impact on performance“. ([opendronemap](#)) Community-Benchmarks zeigen insgesamt nur 5-24 % Gewinn, je nach Datensatz. ([opendronemap](#)) Bei „ultra“ passen 20-MP-Bilder unter Umständen nicht mehr in den VRAM: Ein Nutzer berichtet von 2,1 GB VRAM bei feature-quality „high“ und 7,3 GB bei „ultra“. ([opendronemap](#)) ([opendronemap](#)) Laut Fork-README bietet ODX zusätzlich GPU-Feature-Matching. Das ist eine Selbstausskunft und unabhängig noch nicht geprüft. ([github](#))

Lizenzen und AGPL: ODM, NodeODM und WebODM stehen unter AGPL-3.0.

([aerocartwright](#)) Wenn AeroNexus NodeODM als **unveränderten, separaten Container** nur über HTTP anspricht, entsteht nach gängiger Auslegung kein abgeleitetes Werk. Der eigene Code muss dann nicht unter die AGPL gestellt werden. Werden ODM bzw. NodeODM *modifiziert* und Dritten über das Netz bereitgestellt, muss der geänderte Quelltext angeboten werden. Für ein Open-Source-Projekt ist das unkritisch, wichtig ist nur: Patches in einem eigenen öffentlichen Fork halten und die Lizenzdateien mitliefern. Code aus WebODM (AGPL) darf nicht in die Vue-UI kopiert werden, falls AeroNexus eine andere Lizenz hat. Das ist keine Rechtsberatung.

Alternativen im Vergleich:

- COLMAP + OpenMVS (BSD bzw. AGPL): gute Rekonstruktion, aber keine fertige Orthofoto-, DEM- und Georeferenzierungs-Pipeline.
- Meshroom/AliceVision (MPL-2.0): auf 3D-Modelle ausgerichtet, nicht auf Kartierung.
- MicMac (CeCILL-B): sehr genau, aber steile Lernkurve.
- Kommerziell: DJI Terra, Pix4D und Agisoft Metashape. Sie sind bei Multispektral (Sonnensensor) und Thermal deutlich ausgereifter, dienen hier aber nur als Referenz für die Ergebnisqualität.

ODM bleibt die einzige freie Gesamtlösung mit REST-API, Orthofoto, DSM/DTM, Punktwolke, 3D Tiles und Qualitätsbericht.

Multispektral (M3M): Die ODM-Doku listet „DJI Mavic 3 Multispectral (as of ODM version 3.5.3)“ als unterstützt, mit `--radiometric-calibration camera` bzw. `camera+sun`. Das Ergebnis ist ein N-Band-Orthofoto (plus Alpha), NDVI usw. berechnet man daraus.

([opendronemap](#)) Einschränkungen aus Community und Fachliteratur:

- Die ODM-Kalibrierung basiert auf dem MicaSense-Modell und gilt als experimentell. ([opendronemap](#))

- Die DJI-Sensoren sind nur „relativ“, nicht absolut kalibriert. (opendronemap)
- Nutzer berichten von NDVI-Qualitätslücken gegenüber Pix4Dfields und von Problemen mit (camera+sun). (webodm)

Der „Mavic 3M Image Processing Guide“ von DJI beschreibt die Rohwerte des Sonnensensors und die Formeln (Irradiance, Sensor Gain, Kalibrierparameter (pCam)/(pLS)). (djicdn) **Konsequenz:** NDVI aus ODM eignet sich für relative Vergleiche innerhalb eines Fluges. Für absolute Reflektanz braucht es Kalibrierpanels (Empirical Line) und gegebenenfalls ein eigenes Vorverarbeitungsskript nach dem DJI-Guide. Empfohlene Startoptionen aus der Community: (primary-band:NIR), (radiometric-calibration:camera+sun), (skip-3dmodel:true). (webodm)

Thermal (M3T/M4T R-JPEG): ODM kann das proprietäre DJI-R-JPEG-Format nicht direkt auswerten. Aus einem R-JPEG nutzt ODM nur das 8-Bit-Vorschaubild, die Temperaturen gehen verloren. (opendronemap) (opendronemap) Der bewährte Weg führt über uav4geo „Thermal-Tools“, die das DJI Thermal SDK verpacken und R-JPEGs in 32-Bit-Float-Temperatur-TIFFs umwandeln. (opendronemap) Diese TIFFs gehen dann in ODM ((radiometric-calibration) je nach Datensatz). Laut DJI Download Center ist das DJI Thermal SDK bei v1.8 (2025-12-11, Windows/Linux) und nennt unter „Supported Products“ u. a. „DJI Mavic 3 Enterprise“ und „DJI Matrice 4 Series“. Laut WebODM-Lightning-Blog wurden die Thermal-Tools „updated with the latest DJI SDK, which adds support for the DJI Matrice 4 Series“. Die Binärdateien unterliegen einer DJI-EULA und dürfen nicht ins Repo oder in öffentliche Images. (github) (github) Sie werden beim Bau des Agent-Images lokal eingebunden. Eine ARM64-Version ist nicht belegt, das ist ein weiterer Grund für x64. (github) Die Ergebnisqualität ist begrenzt: 640×512 Pixel, wenig Merkmale, Reflexionen auf PV-Modulen und Artefakte. (opendronemap +3) Zu empfehlen sind 80/80 % Überlappung, niedrige Auflösungsziele (ortho-resolution ≈ GSD) und gegebenenfalls Ausrichtung über das parallel aufgenommene RGB. (opendronemap) (opendronemap)

3. Hardwarebedarf x64 (Mavic 3E, 20 MP)

Die Basis ist die ODM-Tabelle „Minimum RAM needed for N images“: 40 → 4 GB, 250 → 16 GB, 500 → 32 GB, 1500 → 64 GB, 2500 → 128 GB („RAM or RAM + Swap“). (opendronemap) Community-Messungen liegen bei „high/ultra“ deutlich darüber, z. B. 1413 Bilder high/high ≈ 98 GB und 473 Bilder ultra/ultra ≈ 125 GB. (opendronemap)

Bilder	Minimum (Doku)	Empfehlung AeroNexus	Grobe Laufzeit*
200	16 GB	32 GB RAM, 8+ Kerne, 200 GB NVMe	ca. 0,5–1,5 h
500	32 GB	64 GB RAM + 64 GB NVMe-Swap, 12–16 Kerne	ca. 2–4 h
1500	64 GB	128 GB RAM oder 64 GB + Split-Merge ((split) ≈ 400–500), 16+ Kerne, 1 TB NVMe	ca. 6–10 h

*Die Laufzeiten sind eigene Abschätzungen, extrapoliert aus Community-Benchmarks (z. B. 1413 Bilder high/high in 467 min mit GPU). (`opendronemap`) Sie hängen stark von (`feature-quality`), (`pc-quality`) und (`orthophoto-resolution`) ab und müssen am eigenen Datensatz kalibriert werden. GPU: eine NVIDIA RTX ab 8 GB VRAM (12-16 GB für „ultra“). Speicherplatz: etwa das 10- bis 20-Fache der Eingangsdaten als Arbeitsbereich.

4. Jobverteilung Pi → x64

Muster: Pull-Agent statt Push.

1. Die Web-UI legt einen (`mapping_job`) an (MySQL: Status (`QUEUED`), Bildliste als MinIO-Keys, ODM-Optionsprofil, Ziel-CRS).
2. Der x64-Agent ruft alle 15-30 s (`POST /api/mapping/agent/claim`) auf (ausgehend, mTLS oder Token). Der Pi vergibt den Job mit einer **Lease** (z. B. 10 min, verlängert per Heartbeat). Über (`capabilities`) meldet der Agent RAM, GPU und Engine-Version, und der Pi lehnt Jobs ab, die zu groß sind.
3. Der Agent lädt die Bilder über **vorsignierte GET-URLs** direkt aus MinIO. Der Pi gibt dabei keine Dauer-Credentials heraus. Alternativ erhält der Agent temporäre STS-Credentials, die auf das Präfix (`mapping/{jobId}/`) beschränkt sind.
4. Der Agent startet einen Task am lokalen NodeODM (`localhost:3000`), nicht im LAN exponiert) und meldet Fortschritt und Log-Ausschnitte per Heartbeat.
5. Nachbearbeitung auf x64: (`gdal_translate -of COG`) (Orthofoto, DSM, DTM, NDVI), Reprojektion nach EPSG:25832/25833, Raster-Kacheln oder PMTiles, COPC für die Punktwolke, 3D Tiles aus ODM.
6. Upload per **vorsignierter PUT-/Multipart-URLs** nach (`mapping-results/{jobId}/...`), danach (`POST /complete`) mit Manifest (Dateien, Prüfsummen, Bounds, CRS, Qualitätsbericht).
7. Läuft eine Lease ab (Rechner aus oder abgestürzt), setzt der Pi den Job wieder auf (`QUEUED`). NodeODM-Tasks lassen sich über die UUID wieder aufnehmen, solange der Knoten seine Daten behält.

Wake-on-LAN (optional): Der Pi sendet ein Magic Packet (z. B. Python (`wakeonlan`), MAC aus (`.env`)), wenn Jobs warten und kein Agent aktiv ist. Danach wartet er auf den ersten Claim. Ein automatisches Herunterfahren nach X Minuten Leerlauf übernimmt der Agent selbst. Unter Windows Docker Desktop mit WSL2-Backend ist zu prüfen, dass (`--gpus all`) funktioniert und der Agent als Dienst startet.

Sicherheit im LAN:

- NodeODM nie direkt im LAN öffnen, nur (`127.0.0.1`). Zusätzlich NodeODM mit (`--token`) starten.
- Agent ↔ Pi über HTTPS mit eigenem CA-Zertifikat, Agent-Token in (`.env`), Agent-ID-Whitelist.

- MinIO-Zugriff nur über kurzlebige vorsignierte URLs (≤ 1 h) oder präfixbeschränkte STS.
- Rate-Limit und Größenlimits auf den Upload-Endpunkten.
- Keine Secrets in Job-Payloads oder Logs.

5. Bereitstellung und Darstellung

Empfehlung für das MVP: Kacheln vorberechnen statt dynamisch serven. Der x64-Knoten erzeugt aus dem COG Web-Mercator-XYZ-Kacheln (gdal2tiles bzw. die in ODM mitgelieferte Variante) oder ein **Raster-PMTiles**. MinIO bzw. Nginx auf dem Pi liefert sie statisch aus, und das kostet den Pi praktisch keine CPU. In Leaflet wird das ein `L.tileLayer` oder ein PMTiles-Layer (Protomaps bietet eine Leaflet-Bibliothek). `github` Das COG bleibt als Analyse- und Download-Artefakt erhalten.

Dynamisches Kacheln (Phase 3+): TiTiler 2.4.0 hat ein ARM64-Image und taugt für Ad-hoc-Indizes (NDVI-Paletten, Rescaling, Thermal-Paletten) direkt aus den COGs in MinIO. `developmentseed` GeoServer ist für dieses Szenario zu schwergewichtig (Java, viel RAM). Ihn bräuchte man nur, wenn WMS/WFS nach außen nötig wird.

3D und Punktwolken:

- ODM erzeugt über obj2tiles 3D Tiles, die sich in CesiumJS (Apache-2.0) anzeigen lassen. `newreleases`
- Punktwolken werden auf x64 in COPC (LAZ 1.4) umgewandelt, mit den in ODM enthaltenen PDAL- bzw. untwine-Werkzeugen. `github` Im Browser lassen sie sich mit Potree bzw. COPC-fähigen Viewern per HTTP-Range-Requests direkt aus MinIO streamen.
- MinIO braucht dafür CORS mit `Range`- und `Content-Range`-Headern.
- Die 3D-Ansicht sollte eine eigene Route bzw. ein eigenes Panel sein und nicht in der Leaflet-2D-Karte stecken.

Leaflet vs. MapLibre vs. OpenLayers:

Kriterium	Leaflet 1.9.4 (behalten)	MapLibre GL JS v6	OpenLayers
Aufwand jetzt	keiner, Fassade existiert	hoch (AMap-Fassade neu, Annotationen, Zonen)	hoch
Raster-/XYZ-Layer, COG-Vorschau	gut	gut	sehr gut (inkl. COG nativ)
OSM-Vektorkacheln offline (PMTiles)	über protomaps-leaflet (Canvas, begrenzt)	nativ, beste Darstellung	gut
3D-Gelände, Neigung	nein	ja (raster-dem, Globe)	nur 2D
Projektionen EPSG:25832	nur über Proj4Leaflet	nur Web Mercator	voll

Empfehlung: Leaflet bleibt für die Phasen 1–3. In Phase 4 wird MapLibre GL JS geprüft, sobald Vektor-Basiskarte und Gelände wichtig werden. Laut MapLibre-Newsletter vom Juli 2026 ist v6 „barely eight months after v5 landed in December 2025“ erschienen, aktuell ist v6.11.2 (24.09.2026). Die AMap-kompatible Fassade schafft dafür eine gute Abstraktionsschicht. Ein Wechsel lässt sich hinter ihr kapseln, statt ihn quer durch die Vue-Komponenten zu ziehen.

6. Offline-Fähigkeit und amtliche Geodaten

- **OSM offline:** `go-pmtiles` (BSD-3-Clause, v1.31.2, ARM64-Binary) schneidet mit `pmtiles extract https://build.protomaps.com/<YYYYMMDD>.pmtiles de.pmtiles --region=deutschland.geojson --maxzoom=15` einen Deutschland-Ausschnitt aus dem Protomaps-Planet-Build (ODbL, „© OpenStreetMap contributors“). `github +4` Die Datei liegt in MinIO und wird per Range-Request ausgeliefert. Das Bauen geschieht einmalig auf x64, das Ausliefern übernimmt der Pi. OpenMapTiles ist eine Alternative mit eigenem Schema, bringt hier aber keinen Mehrwert.
- **basemap.de (BKG):** WMTS-Capabilities unter `https://sgx.geodatenzentrum.de/wmts_basemapde/1.0.0/WMTSCapabilities.xml`, Layer Farbe/Grau in EPSG:3857, 25832 und 25833, Lizenz CC BY 4.0, Quellenvermerk „© GeoBasis-DE / BKG (Jahr) CC BY 4.0“. `geodatenzentrum +2` Es gibt auch eine Vektorvariante (basemap.de Web Vektor), passend zu MapLibre. `bund` `basemap` Für den Offline-Betrieb sollten Kacheln nur für das Einsatzgebiet und im Rahmen der Lizenz vorgehalten werden.
- **Länderdaten:** Seit 2023/2024 stellen die Länder DOP, DGM und weitere Daten weitgehend als Open Data bereit; die Bayerische Vermessungsverwaltung bietet laut GeodatenOnline Bayern „seit 01.01.2023“ DGM1 (1-m-Gitter, Quellenvermerk „Datenquelle: Bayerische Vermessungsverwaltung - www.geodaten.bayern.de“) und DOP40 kostenfrei unter CC BY 4.0 an. Lizenzen und Formate unterscheiden sich je

Land (CC BY 4.0, dl-de/by-2.0, dl-de/zero-2.0). **Konsequenz:** Eine Layer-Registry pflegen (`layers.yaml`): URL, CRS, Lizenz, Attribution, Offline-Erlaubnis). Das DGM1 des Einsatzgebiets wird auf x64 nach COG konvertiert und dient für (a) Geländefolge-Planung bzw. DSM-Import, (b) Höhenprofile und (c) Volumen gegen die Urgeländehöhe.

7. Messwerkzeuge, Georeferenzierung und Koordinatensysteme

- **Strecke und Fläche:** Geodätisch auf dem Ellipsoid rechnen (z. B. geographiclib bzw. Turf) oder in UTM (EPSG:25832/25833). Den Unterschied zur Web-Mercator-Messung muss die UI anzeigen.
- **Höhenprofil:** Das Backend liest die Linie fensterweise aus dem DSM- bzw. DTM-COG (rasterio, GDAL). Bei kleinen Gebieten reicht dafür der Pi, große Profile übernimmt x64.
- **Volumen:** Polygon über dem DSM mit Basisfläche als Ebene, Polygonrand-Interpolation oder DTM bzw. amtlichem DGM1. Ausgabe von Auftrag, Abtrag und Netto, mit Angabe von Auflösung und Unsicherheit.
- **GCP:** ODM nimmt eine (`gcp_list.txt`) mit Projektionskopf (z. B. EPSG:25832) entgegen. Die UI bekommt einen GCP-Editor (Punkt im Bild markieren). Das ist aufwendig und gehört deshalb in Phase 3. Ergebnis ist der ODM-Qualitätsbericht (`report.pdf`) und (`stats.json`) als Genauigkeitsbericht.
- **RTK/PPK beim M3E:** Mit RTK-Modul stehen hochgenaue Positionen in den EXIF/XMP-Daten. ODM sollte dann mit kleiner (`gps-accuracy`) laufen. PPK (Nachprozessierung der Rohbeobachtungen) leistet ODM nicht. Dafür wären externe Werkzeuge nötig (z. B. RTKLIB), um die Bildkoordinaten zu korrigieren und als (`geo.txt`) zu übergeben.
- **CRS:** ODM gibt standardmäßig UTM auf WGS84 aus. Für amtliche Nutzung wird mit (`gdalwarp -t_srs EPSG:25832`) umprojiziert. Der Unterschied zwischen WGS84 (Realisierung über ITRF) und ETRS89 beträgt in Deutschland inzwischen grob 0,8-0,9 m. Ohne RTK-Bezug zu SAPOS liegt das unter der GNSS-Unsicherheit, mit RTK ist es relevant. **Höhen:** DJI-EXIF-Höhen sind ellipsoidisch oder EGM96. DHHN2016 (NHN) erhält man über das German Combined Quasigeoid GCG2016 per PROJ-Grid. Ob das Grid in PROJ frei verfügbar ist, ist vor der Umsetzung zu klären. Bis dahin werden Höhen klar als „ellipsoidisch“ gekennzeichnet.

Empfohlene Zielarchitektur

Raspberry Pi 5 (ARM64, bestehender Compose-Stack, ergänzt um):

- (`backend`) (Spring Boot) mit neuem Modul (`mapping`): REST für Jobs, Agent-Claim/Heartbeat/Complete, Layer-Registry, Mess-Endpunkte. Flyway-Migrationen für (`mapping_job`), (`mapping_job_event`), (`mapping_result`), (`map_layer`).
- (`minio`): Buckets (`media`) (vorhanden), (`mapping-results`), (`basemaps`) (PMTiles), CORS für Range-Requests.

- `web` (Vue 3): Seiten „Mapping-Jobs“ und „Ergebnisse/Layer“. Leaflet-Layer für Ergebnis-Kacheln und Offline-Basiskarte.
- Optional ab Phase 3: `titiler` (ARM64-Image) und ein `wol`-Helfer.

x64-Knoten (Linux oder Windows/WSL2, Docker, optional NVIDIA Container Toolkit):

- `compute-agent` (Python 3.12, eigener Code, Apache-2.0 bzw. Projektlizenz): Claim, Download, Steuerung von NodeODM, GDAL-Nachbearbeitung, Upload.
- `nodeodm` (AGPL, unverändert oder als eigener reproduzierbarer GPU-Build), nur an localhost gebunden.
- Optional: ein `thermal-prep`-Container mit lokal eingebundenem DJI Thermal SDK (nicht veröffentlicht).

Datenfluss: Flug (Pilot 2) → automatischer Medien-Upload nach MinIO (`media/`) → Bildauswahl in der UI (nach Flug, Zeitraum oder Polygon) → `mapping_job` (QUEUED) → optional WOL → Agent claimt den Job → vorsignierter Download → NodeODM → COG, Kacheln, COPC, 3D Tiles und Bericht → vorsignierter Upload nach `mapping-results/{jobId}/` → `complete` mit Manifest → Backend legt `map_layer` an → die UI zeigt den Layer (Transparenz-Regler, Vergleich), Messwerkzeuge greifen auf die COGs zu.

Komponentenliste (zu pinnen)

Komponente	Version (Pin)	Lizenz	ARM64 / x64	Ort
ODM (Engine)	3.6.1 bzw. 3.6.2, <code>opendronemap/odm:3.6.1-gpu</code> per Digest	AGPL-3.0	x64 (ARM64-Builds existieren, für ODM auf dem Pi aber ungeeignet)	x64 (<code>github</code>)
NodeODM	3.6.2 (GPU-Image selbst bauen, Issue #271)	AGPL-3.0	x64	x64
Alternative: ODX/NodeODX	NodeODX 2.3.1, <code>webodm/nodeodx</code> per Digest	AGPL-3.0	x64	x64 (<code>github</code>) (<code>docker</code>)
PyODM (optional im Agenten)	aktuelle Version auf PyPI, pinnen	BSD-3 (laut Repo prüfen)	beide	x64
GDAL	3.11.x (wie in ODM 3.6.1)	MIT	beide	x64 (+ Pi für Messungen)
DJI Thermal SDK	v1.8 (2025-12-11)	proprietär (DJI-EULA)	nur x86/x64	x64 (<code>dji</code>)
uav4geo Thermal-Tools	Commit-Hash pinnen	Lizenz im Repo prüfen	x64	x64
TiTiler	2.4.0	MIT	ARM64-Image vorhanden	Pi (optional) (<code>developmentseed</code>)
go-pmtiles	v1.31.2	BSD-3-Clause	beide	x64 (Bau), Pi (optional Serve) (<code>github</code>) (<code>github</code>)
Protomaps-Basemap-Daten	Build-Datum pinnen	ODbL	-	MinIO
Leaflet	1.9.4 (vorhanden)	BSD-2-Clause	Browser	Web

Komponente	Version (Pin)	Lizenz	ARM64 / x64	Ort
MapLibre GL JS	v6.x (Phase 4, exakt pinnen)	BSD-3-Clause	Browser	Web
CesiumJS / Potree	bei Einführung pinnen	Apache-2.0 / BSD-2	Browser	Web
basemap.de WMTS	Dienst	CC BY 4.0	-	online/Cache (bund)

Hinweis: Versionsnummern von PyODM, Potree, CesiumJS und Thermal-Tools wurden in dieser Recherche nicht verifiziert. Sie sind bei Einführung aus dem jeweiligen Release zu übernehmen.

Risiken und offene Punkte

- WPML-Kompatibilität (hoch):** M4E/M4T-Enums sind nur sekundär belegt (Payload 88 gar nicht). [github](#) [github](#) [mappingPrism](#) und [mappingCylinder](#) sind undokumentiert. Fassade und Echtzeit-Geländefolge sind laut Spezifikation M3-exklusiv. [github](#) Gegenmaßnahmen: Golden Files aus Pilot 2 je Modell, im MVP nur Import und Anzeige, ein eigener Generator erst danach, und Flugtests mit sicherem Profil.
- Spaltung ODM/ODX (mittel):** Beide Linien können auseinanderlaufen. [aerocartwright](#) Daher nur die API-Schnittstelle nutzen, Digests pinnen und Upgrades bewusst testen.
- GPU-Images (mittel):** Wie in Issue #271 kann die CPU-Ausweichlösung unbemerkt greifen. Deshalb nach dem Start im Agenten die GPU prüfen ([has_popsift_and_can_handle_texsize](#)) bzw. Log-Eintrag „Using GPU for extracting SIFT features“. [opendronemap](#) [github](#)
- Multispektral (mittel/hoch):** Die radiometrische Kalibrierung in ODM ist experimentell. [opendronemap](#) Ergebnisse als „relativer Index“ kennzeichnen und Panel-Workflow einplanen.
- Thermal (hoch):** Das SDK ist proprietär und nicht weitergebar. [github](#) Die Orthofotos zeigen häufig Artefakte, [opendronemap](#) [opendronemap](#) die Temperaturgenauigkeit hängt von Emissivität, Distanz und Luftfeuchte ab.
- AGPL (niedrig bei sauberer Trennung):** Keine Code-Übernahme aus WebODM, NodeODM nur als separater Dienst und Patches veröffentlichen.
- Pi-Ressourcen (mittel):** Große COGs und PMTiles auf der SD-Karte sind problematisch. NVMe/USB-SSD für MinIO ist Pflicht, und rechenintensive Schritte bleiben auf x64.
- Höhenbezug DHHN2016 (mittel):** Verfügbarkeit des GCG2016-Grids klären.

Recommendations - Priorisierte Aufgabenliste für Claude Code

Phase 0 - Grundlagen (0,5-1 Woche)

- ☐ ADR anlegen: „Pull-Agent + NodeODM-API, Engine-Linie ODM 3.6.x gepinnt, ODX als Alternative“.
- ☐ `.env.example` erweitern: `MAPPING_AGENT_TOKEN`, `MAPPING_RESULTS_BUCKET`, `WOL_MAC`, `WOL_BROADCAST`, `PRESIGN_TTL_SECONDS`.
- ☐ MinIO: Bucket `mapping-results` und `basemaps` per Init-Skript anlegen, CORS mit `Range`/`Content-Range` setzen.

Phase 1 - MVP: Orthofoto aus MinIO-Bildern (2-3 Wochen)

- ☐ Backend: Flyway-Migration für `mapping_job` (id, status, image_keys JSON, options JSON, lease_until, agent_id, progress, error), `mapping_result` und `map_layer`.
- ☐ Backend: Endpunkte `POST/GET /api/mapping/jobs`, `POST /api/mapping/agent/claim|heartbeat|complete|fail` mit Token-Auth, Lease-Logik und Rückfall auf QUEUED.
- ☐ Backend: Service für vorsignierte GET-URLs (Eingangsbilder) und PUT/Multipart-URLs (Ergebnisse), präfixbeschränkt.
- ☐ Neues Repo-Verzeichnis `compute-agent/` (Python 3.12, `requirements.txt` gepinnt): Claim-Schleife, paralleler Download, NodeODM-Client (init/upload/commit/info/download), Fortschritt per Heartbeat.
- ☐ Agent-Nachbearbeitung: `gdal_translate -of COG` für Orthofoto und DSM, `gdalwarp` nach EPSG:25832 (konfigurierbar), Web-Mercator-Kacheln (`gdal2tiles`, Zoom bis GSD-Äquivalent), `manifest.json` (Bounds WGS84, CRS, Dateien, SHA-256).
- ☐ `compute-agent/docker-compose.yml` für x64: `nodeodm` (gepinnter Digest, `127.0.0.1:3000`, `--token`) und `agent`. GPU-Variante als Override-Datei.
- ☐ Web-UI: Seite „Mapping-Jobs“ (Bildauswahl nach Flug bzw. Datum, Optionsprofil „schnell/standard/hoch“, Statusliste) und Layerliste mit Leaflet-`tileLayer`, Transparenz und Zoom auf das Ergebnis.
- ☐ Akzeptanztest: ODM-Testdatensatz und ein eigener M3E-Flug mit etwa 200 Bildern, Orthofoto in der Karte, Lage gegen DOP geprüft.

Phase 2 - Robustheit und Betrieb (1-2 Wochen)

- ☐ Wake-on-LAN-Helfer am Pi, Leerlauf-Shutdown im Agenten (konfigurierbar).
- ☐ GPU-Health-Check im Agenten und Ausweisung von „GPU aktiv: ja/nein“ in der UI.
- ☐ Wiederaufnahme: Agent merkt sich NodeODM-UUID je Job, bei Neustart Status abfragen statt neu rechnen.

- ☐ Ressourcenprüfung: Agent meldet RAM, Pi verweigert Jobs über der Kapazität oder schlägt `split` vor.

Phase 3 – Messen und Genauigkeit (2–3 Wochen)

- ☐ DSM/DTM-Layer mit Farbverlauf und Schummerung (vorberechnet auf x64), Download der COGs.
- ☐ Messwerkzeuge: geodätische Strecke und Fläche, Höhenprofil (Backend liest COG), Volumen (Basis Ebene/Rand/DTM).
- ☐ GCP-Workflow: Import von Passpunkten (CSV, EPSG:25832), `gcp_list.txt`-Erzeugung, Anzeige von `report.pdf` und `stats.json`.
- ☐ RTK-Flag pro Datensatz und automatische Absenkung von `gps-accuracy`.
- ☐ Optional: TiTiler-Container auf dem Pi für dynamische Paletten.

Phase 4 – Karten offline und amtlich (1–2 Wochen)

- ☐ Skript `tools/build-basemap.sh` (x64): `pmtiles extract` für Deutschland bzw. das Einsatzgebiet, Upload nach `basemaps/`.
- ☐ `layers.yaml`-Registry (basemap.de WMTS, Länder-DOP-WMS, Attribution, Lizenz, Offline-Flag) und Layer-Umschalter in der UI.
- ☐ DGM1-Import (Einsatzgebiet) als COG für Profile, Volumen und Planung.
- ☐ Prototyp mit MapLibre GL JS hinter der bestehenden Fassade, Entscheidung zur Migration per ADR.

Phase 5 – Flugplanung WPML (3–4 Wochen, mit Feldtests)

- ☐ Golden-File-Korpus: echte Pilot-2-KMZs je Modell (M3E, M3T, M3M, später M4E/M4T) und Templatetyp ablegen.
- ☐ Parser und Validator für `template.kml`/`waylines.wpml` (sicherer XML-Parser, unbekannte Elemente erhalten).
- ☐ Generator mapping2d (Polygon, Überlappung, Richtung, Rand, Höhe, `elevationOptimizeEnable`, `smartObliqueEnable`) mit GSD-Rechner und Kamera-Profil-JSON. Danach mappingStrip und mapping3d. Fassade nur für M3E/M3T/M3M.
- ☐ Geländefolge: DSM- bzw. DGM1-Ausschnitt bereitstellen. Ob Pilot 2 ein eigenes DSM aus dem KMZ (`res/`) übernimmt, durch Golden Files klären.
- ☐ Upload in die bestehende Wegpunkt-Bibliothek, Feldtest mit sicherem Profil vor Freigabe.

Phase 6 – Multispektral, Thermal und 3D (je 2–3 Wochen)

- ☐ M3M-Profil: `radiometric-calibration=camera+sun`, `primary-band=NIR`, Ausgabe von NDVI/NDRE als COG mit Kennzeichnung „relativ“, optional Panel-Kalibrierung.

- ❑ Thermal: [thermal-prep](#)-Container (SDK lokal eingebunden), R-JPEG → Float-TIFF, eigenes ODM-Profil, Paletten-Darstellung.
- ❑ 3D: COPC aus der Punktwolke, 3D Tiles aus ODM, eigene 3D-Ansicht (CesiumJS oder Potree), MinIO-Range-Streaming.

Caveats

- Mehrere Angaben sind nicht aus Primärquellen bestätigt: die M4E/M4T-Enums (99/0, 99/1, 89) und vor allem der M4E-Kamera-Enum 88. Die Live-Doku von DJI war technisch nicht vollständig auslesbar. Belastbar sind die Werte für M3E/M3T/M3M aus dem DJI-Doku-Repo. [github](#)
- Die Laufzeitschätzungen sind Extrapolationen und keine Messungen. Die RAM-Tabelle von ODM nennt Mindestwerte, die bei hohen Qualitätsstufen deutlich überschritten werden.
- Die ODX-Angaben (z. B. „faster than ODM“, GPU-Matching) sind Selbstauskünfte des Forks. [github](#) Einige Erklärseiten zur Spaltung stammen von Drittanbietern. Belastbar sind die WebODM-Ankündigung und die GitHub-Repos.
- Die Aussagen zur AGPL sind eine technische Einschätzung und keine Rechtsberatung.
- Der Datumsversatz WGS84/ETRS89 (~0,8–0,9 m) ist ein Näherungswert, für Vermessungszwecke gelten die SAPOS- bzw. AdV-Vorgaben.